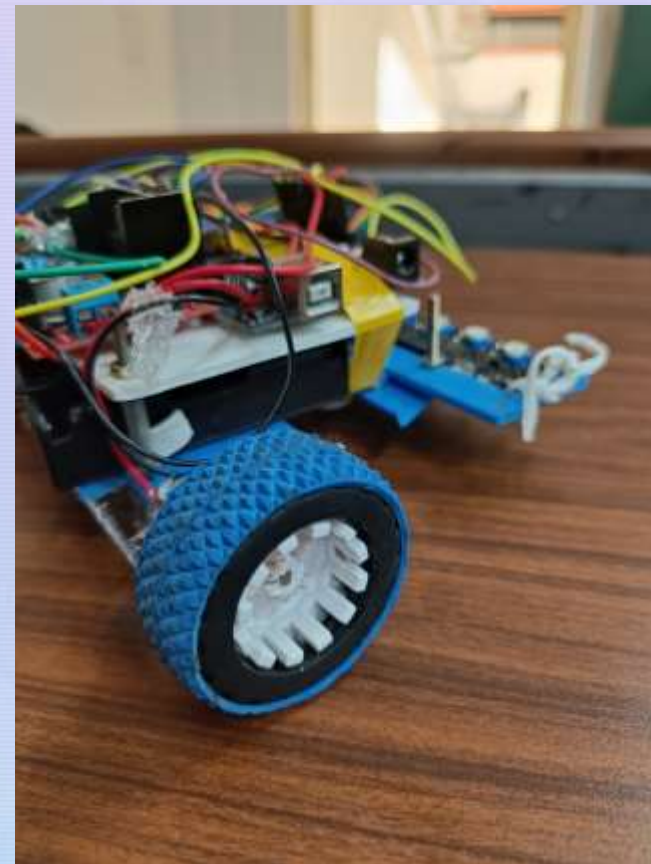


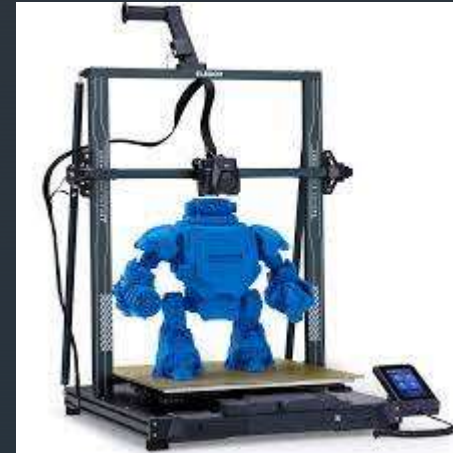
Fast Line follower Robot

Devashish Sharma



▀ Fabrication

- Acrylic sheets (CNC machine cutting)
- Polylactic acid (PLA)
(3D printer)



Control Theory

Sensor-to-position mapping

The QTR library reads 6 IR sensors and computes a weighted average position:

$$\text{position} = \frac{\sum_{i=1}^6 s_i \cdot w_i}{\sum_{i=1}^6 s_i}$$

where s_i is the reflectance reading of sensor i and $w_i \in \{0, 1000, 2000, 3000, 4000, 5000\}$ is its positional weight. The result is a value from **0 to 5000**, representing the line's lateral position across the sensor array. The center is 2500.

Error signal

The robot defines its reference (setpoint) as the line being directly centered under the sensor array:

$$e(t) = \text{position}(t) - 2500$$

$e > 0 \rightarrow$ line is to the right $\rightarrow$ steer right (slow left motor, speed up right)

$e < 0 \rightarrow$ line is to the left $\rightarrow$ steer left

$e = 0 \rightarrow$ robot is perfectly on the line

PD controller

Your controller is a **Proportional-Derivative (PD)** control law, not full PID, because an integral term would cause windup and overshoot on a fast robot:

$$u(t) = K_p \cdot e(t) + K_d \cdot \frac{\Delta e}{\Delta t}$$

Since the loop runs at fixed discrete timesteps, the derivative becomes a simple finite difference:

$$u(t) = K_p \cdot e(t) + K_d \cdot (e(t) - e(t - 1))$$

This is exactly what your code computes:

```
int motorSpeed = Kp * error + Kd * (error - lastError);
```

Role of each term:

$K_p = 0.0005$ (proportional): scales the raw correction.

Note: Small value because the raw error can reach ± 2500 , and at full base speed (210) even a modest motorSpeed swing matters. If K_p is too large $\rightarrow$ oscillation; too small $\rightarrow$ sluggish tracking.

$K_d = 0.092$ (derivative): damps the response by opposing *changes* in error.

Note: When the error is growing fast (approaching a curve), the D term adds extra correction before the error even gets large. This is what allows high-speed operation without fishtailing.

Motor speed mixing

The control output u is applied differentially across the two motors:

$$v_R = v_{base} + u \mid v_L = v_{base} - u$$

This is a classic **differential drive** mixing strategy. When the robot drifts right ($e > 0, u > 0$):

right motor speeds up → pulls nose back left

left motor slows down → same effect

Both motors together keep average forward velocity $\approx \text{BaseSpeed} = 210$.

Clamping prevents negative or over-limit PWM values: $v \in [0, \text{MaxSpeed}]$

Hard-turn override (bang-bang fallback)

When $\text{position} < 300$ or $\text{position} > 4700$, the line is almost entirely off one end of the sensor array. In this regime the proportional correction alone is too slow, so the code **bypasses the PD loop entirely** and commands a full counter rotation at $\text{speedturn} = 180$.

Why no integral term?

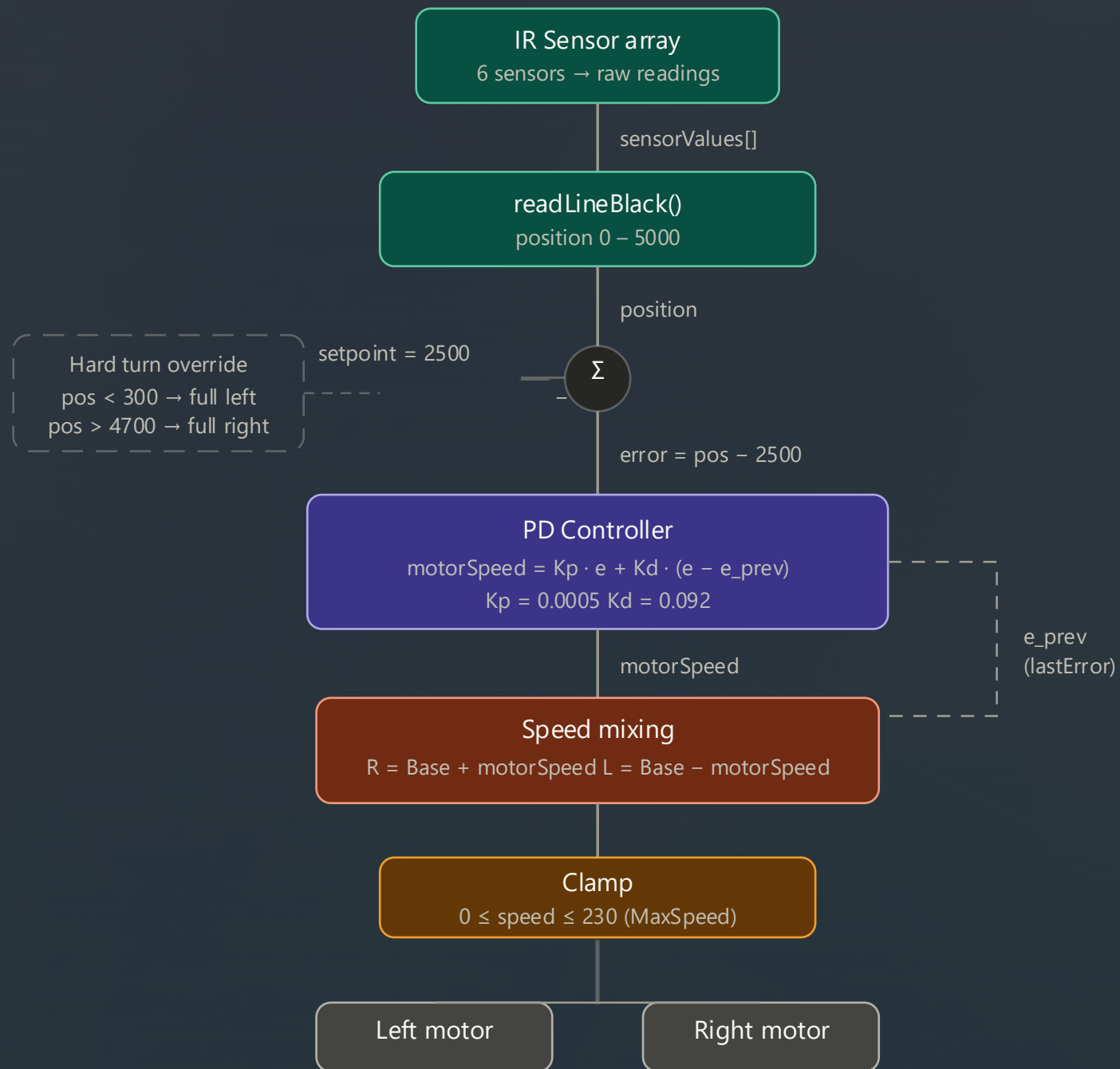
A PID's I term integrates accumulated error to correct steady-state offsets. In line following:

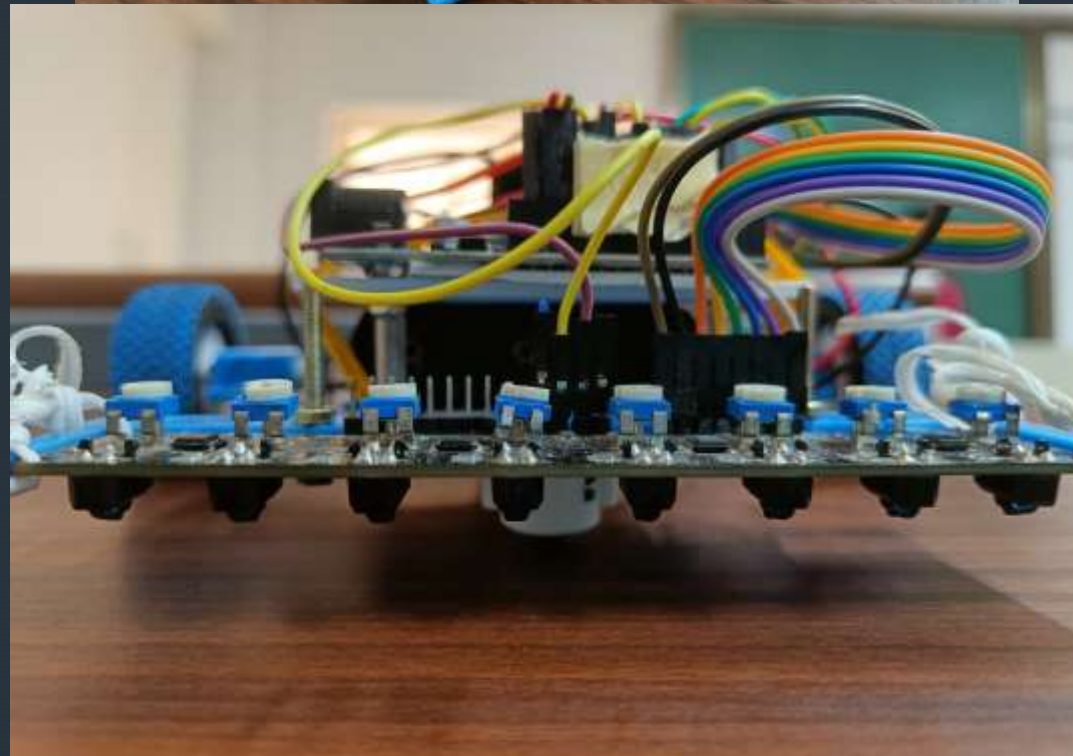
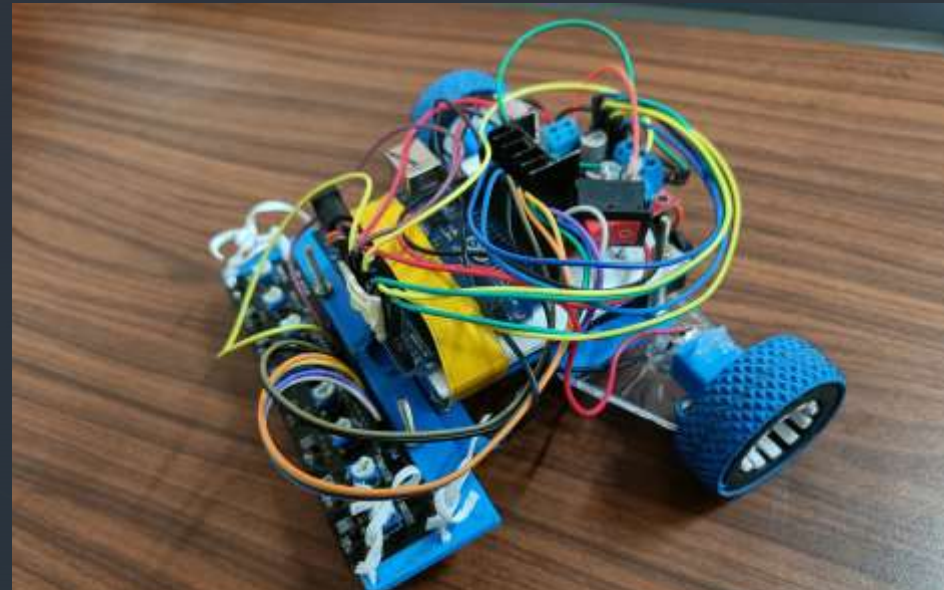
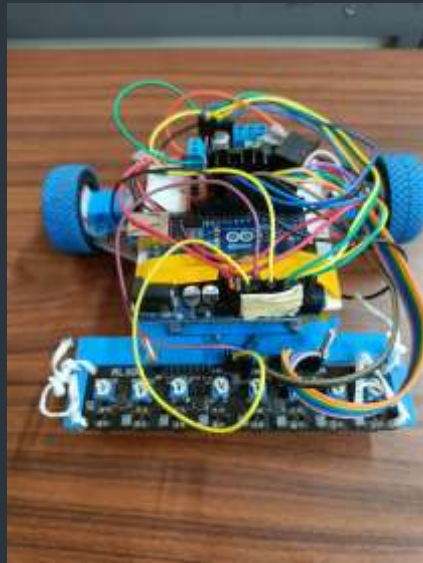
There's no persistent steady-state error (the track keeps changing)

Integral windup makes fast direction changes sluggish

The D term already handles the dynamic correction

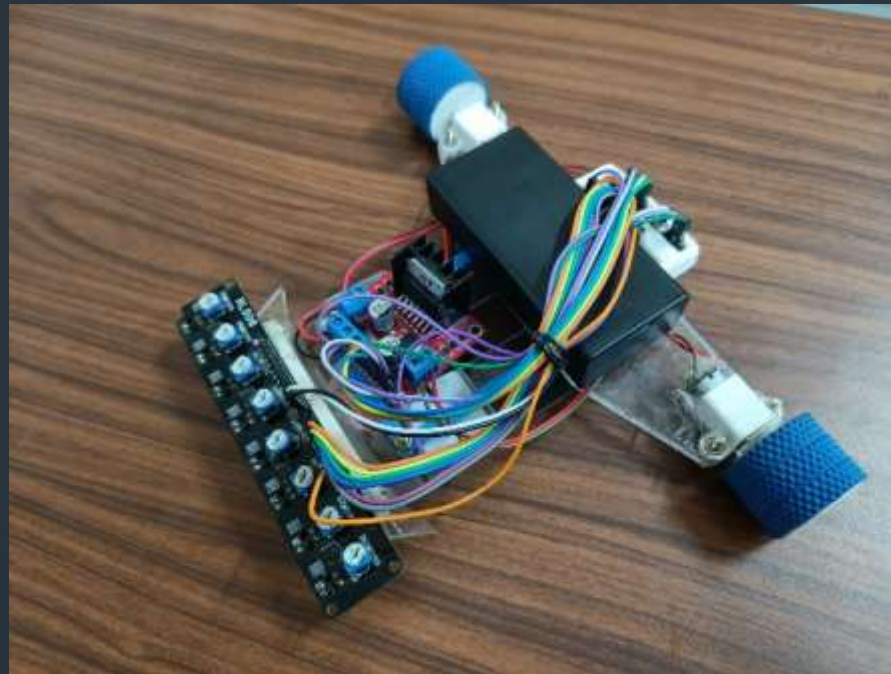
Block diagram





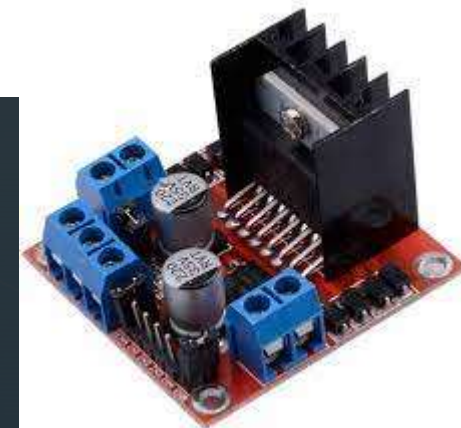
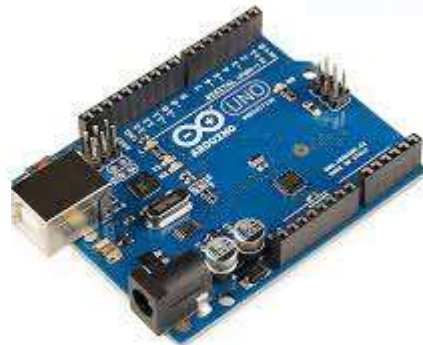
Version 2

- Smoother and steadier motion
- Aerodynamic design to increase performance





Components used



THANK YOU!

